

METODOLOGÍA DE LA PROGRAMACIÓN

TEMA 8: PRUEBAS SOFTWARE



Objetivos

- ✓ Introducir el concepto de prueba software
- ✓ Implementar pruebas unitarias con JUnit
- ✓ Mostrar el concepto de sistema de integración continua

Índice: Tema 6

6.1 Introducción

6.1.1 Conceptos básicos

6.1.2 Tipos de pruebas

6.2 Pruebas unitarias

6.2.1 Concepto

6.2.2 Framework JUnit

6.3 Sistemas de integración continua

6.3.1 Introducción

Índice: Tema 6

6.1 Introducción

6.1.1 Conceptos básicos

6.1.2 Tipos de pruebas

6.2 Pruebas unitarias

6.2.1 Concepto

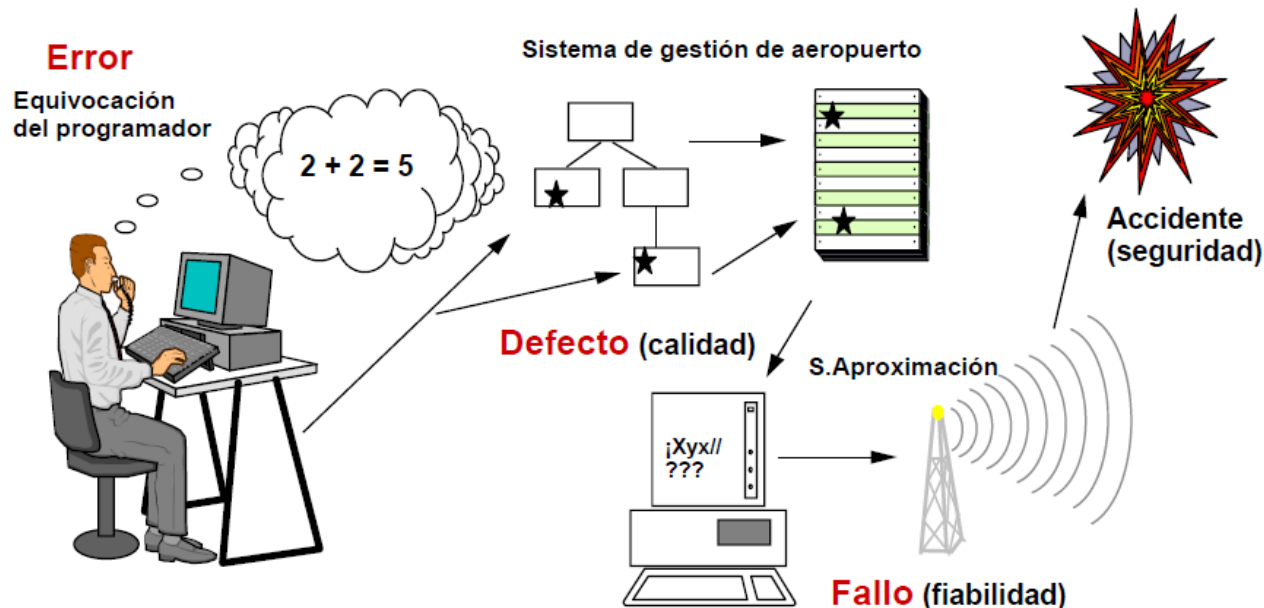
6.2.2 Framework JUnit

6.3 Sistemas de integración continua

6.3.1 Introducción

➤ Relación entre error, defecto y fallo

- ✓ **Error:** Acción humana que lleva a un resultado incorrecto.
- ✓ **Defecto:** Un paso de procesamiento incorrecto en el programa.
- ✓ **Fallo:** Es la incapacidad de un sistema para realizar las funciones requeridas dentro de los requisitos de rendimiento especificados.



- Un **error** del programador introduce un **defecto** y este defecto produce un **fallo**.
- La manera adecuada de enfrentarse al reto de la **calidad** es la **prevención**.
- **Pruebas:**
 - ✓ Las pruebas son técnicas de **comprobación dinámica**:
 - Siempre implican la **ejecución** del programa.
 - ✓ Permiten:
 - Evaluar la calidad de un producto.
 - Mejorarlo identificando defectos y problemas.

- El **objetivo** en el diseño de pruebas es “romper” el programa, es decir, encontrar el mayor número de fallos posible.
- Una prueba tiene éxito si descubre un nuevo error.
- **Una prueba no asegura la ausencia de defectos, sólo puede demostrar que existen errores en el software.**
- Las pruebas deberán planificarse mucho antes de que empiecen.
- Empezar por lo pequeño y progresar hacia lo grande.
- No son posibles las pruebas exhaustivas

- El principio de Pareto es aplicable a las pruebas del software:
 - El 80% de los errores está en el 20% de los módulos.
 - Hay que identificar y probar bien esos módulos.

- *Durante las fases anteriores de desarrollo, el ingeniero intenta **construir**, sin embargo, durante la fase de pruebas se crean una serie de casos de prueba que intentan **demoler** el software construido. La intencionalidad es encontrar fallos.*

➤ **Pruebas** (*test*):

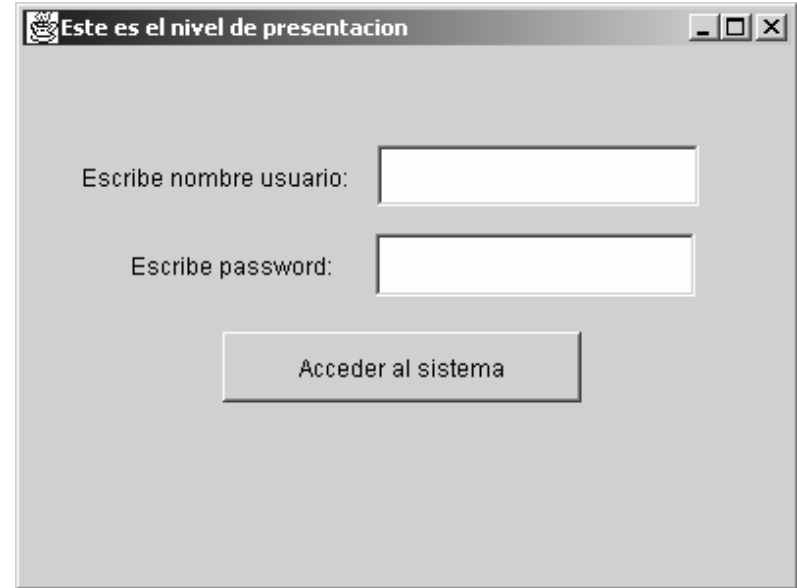
- ✓ Es una actividad en la cual un sistema o uno de sus componentes se ejecuta en circunstancias previamente especificadas, los resultados se observan y registran y se realiza una evaluación de algún aspecto.

➤ **Caso de Prueba** (*test case*):

- ✓ Es un conjunto de entradas (escenario de prueba), condiciones de ejecución y resultados esperados.
- ✓ Tiene un objetivo concreto (probar algo).

➤ Ejemplo de caso de prueba:

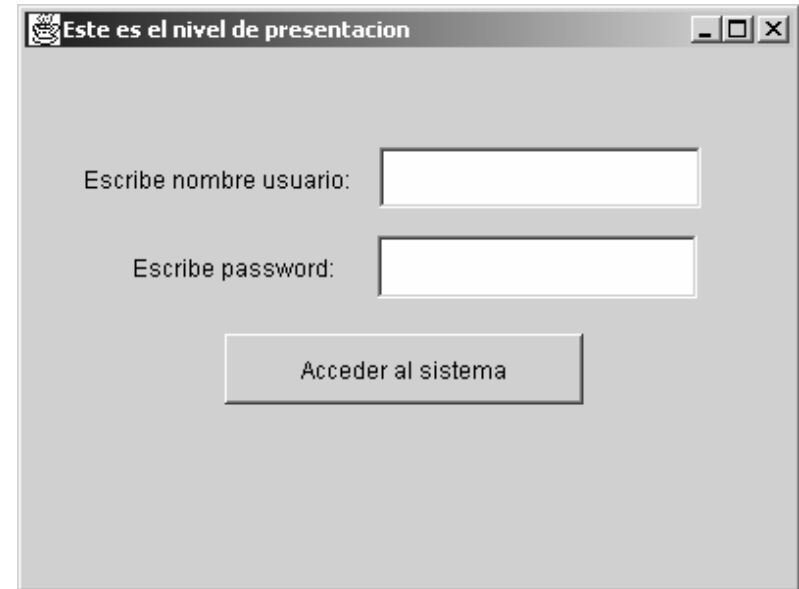
- Caso de prueba para caso de uso de “Acceder al Sistema”.
- **ENTRADA:**
 - **Nombre de usuario:** Hacker.
 - **Password:** 0000
- **CONDICIONES DE EJECUCIÓN:**
 - No existe en el sistema (en la base de datos) la tupla “Hacker, 0000”.
 - Pero sí existe “Hacker, 1111”.
- **RESULTADO ESPERADO:**
 - No deja entrar al sistema.
- **OBJETIVO DEL CASO DE PRUEBA:**
 - Comprobar que no deja entrar a un usuario que sí existe pero que introduce un password equivocado.



➤ Ejemplo de caso de prueba:

- Caso de prueba para caso de uso de “Acceder al Sistema”.

- ¿Cuál sería el procedimiento de la prueba?
 - Ejecutar la clase de presentación.
 - Introducir los datos de la prueba (Usuario “hacker”, password “0000”).
 - Pulsar botón “Acceder al sistema”.
 - Comprobar que no ha accedido.



➤ **Componente de prueba:**

- Programa que automatiza la ejecución de uno (o varios) casos de prueba.
- Una vez escrito, se puede probar muchas veces (cada vez que haya un cambio en el código de una clase que pueda afectarle).

Índice: Tema 6

6.1 Introducción

6.1.1 Conceptos básicos

6.1.2 Tipos de pruebas

6.2 Pruebas unitarias

6.2.1 Concepto

6.2.2 Framework JUnit

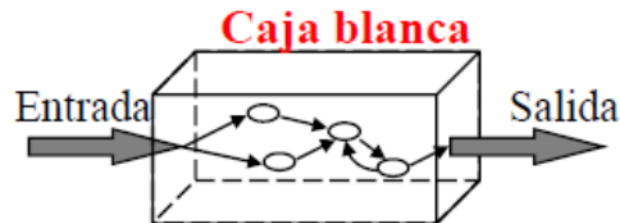
6.3 Sistemas de integración continua

6.3.1 Introducción

- Hemos dicho que el **objetivo** en el diseño de pruebas encontrar el mayor número de fallos posible.
- Interesa diseñar los casos de prueba que tengan la mayor **probabilidad** de encontrar el mayor número de errores con la mínima cantidad de esfuerzo y tiempo.
- Para ello, existen dos enfoques:
 - ✓ Pruebas de **caja blanca**:
 - Prueba Estructural.
 - Basadas en información sobre cómo el software ha sido diseñado o codificado.
 - ✓ Pruebas de **caja negra**:
 - Prueba Funcional.
 - Los casos de prueba se basan sólo en el comportamiento de entrada/salida.

➤ Pruebas de caja blanca

- ✓ Denominada a veces prueba de caja de cristal
 - Aseguran que la **operación interna** del programa se ajusta a las especificaciones y que todos los componentes internos se han probado adecuadamente.
 - Usa la **estructura de control** para obtener los casos de prueba.
 - Intentan garantizar que **todos los caminos de ejecución** del programa quedan probados.



➤ Pruebas de caja blanca

- ✓ El ingeniero del software debe diseñar casos de prueba que:
 1. Garanticen que se ejecuta **por lo menos una vez** todos los caminos independientes de cada módulo.
 2. Evalúen **todas las decisiones lógicas** en sus vertientes verdadera y falsa.
 3. Ejecuten **todos los bucles en sus límites** y con sus límites operacionales. Se diseñan casos de prueba para que se intente ejecutar un bucle 0,1,...,n-1,n y n+1 veces (siendo n el número máximo)
 4. Ejerciten las **estructuras internas** de datos para asegurar su validez.

➤ Pruebas de caja blanca

Ejemplo.

```
public class Esprimo {

    public static boolean esPrimo (String args[])
        throws    ErrorFaltaParametro,
                  ErrorSololParametro,
                  ErrorNoNumeroPositivo {
        if (args.length == 0) throw new ErrorFaltaParametro();
        else if (args.length > 1) throw new ErrorSololParametro();
        else {
            try {
                float numF = Float.parseFloat(args[0]);
                int num = (int)numF;
                if (num<=0) throw new ErrorNoNumeroPositivo();
                else {
                    for (int i=2;i<num;i++)
                        if (num%i==0) {return false;}
                    return true;
                }
            }
            catch (NumberFormatException e) { throw new ErrorNoNumeroPositivo(); }
        }
    }
}
```


➤ Pruebas de caja blanca

Ejemplo.

NUM	SalidaEsper	comentario
	falta parámetro	if (args.length == 0) => EJECUTAR "then" // (args.length)==true
xx yy	sólo 1 param	if (args.length > 1) => EJECUTAR "then" // (args.length)==false // (args.length > 1)==true
-4	no positivo	if (num<=0) => EJECUTAR "then" // (args.length > 1)=false // (num<=0)==true
2	primo	for (int i=2;i<num;i++) EJECUTAR BUCLE 0 VECES // (num<=0)==false
3	primo	for (int i=2;i<num;i++) EJECUTAR BUCLE 1 VEZ
4	no primo	for (int i=2;i<num;i++) EJECUTAR 2 VECES // if (num%i==0) => EJECUTAR "then" // (num%i==0)==true
23	primo	for (int i=2;i<num;i++) EJECUTAR BUCLE N VECES // (num%i==0)==false
patata	no positivo	Probar la condición (excepción) catch (NumberFormatException e)

OBJETIVO A PROBAR

- Probar todos los caminos
- Probar todas las condiciones
- Probar bucles

➤ Pruebas de caja blanca

Ejemplo.

¿Componente de prueba?

➤ Pruebas de caja blanca

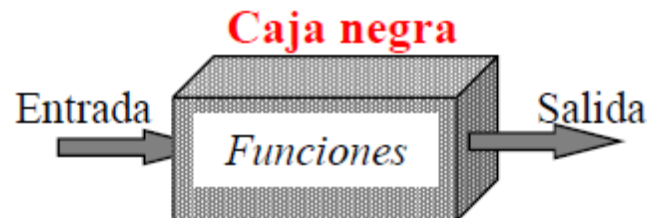
```
public class ComponentePruebaEsPrimo {
    public static void main(String[] args) {
        String[] s1 = new String[0];
        try {
            boolean b1 = Esprimo.esPrimo(s1);
            System.out.println("CP1 incorrecto");
        } catch (ErrorFaltaParametro e){
            System.out.println("CP1 correcto");
        } catch (Exception e) {
            System.out.println("CP1 incorrecto");
        }
        String[] s2 = new String[2]; s2[0]="xx"; s2[1]="yy";
        try {
            boolean b1 = Esprimo.esPrimo(s2);
            System.out.println("CP2 incorrecto");
        } catch (ErrorSolo1Parametro e){
            System.out.println("CP2 correcto");
        } catch (Exception e) {System.out.println("CP2 incorrecto");
        } ...
    }
}
```

Esto prueba una
entrada sin
parámetro

Esto prueba una
entrada con dos
parámetros

➤ Pruebas de caja negra

- ✓ Denominada a veces prueba de comportamiento:
 - Se centran en los requisitos **funcionales** del software.
 - El ingeniero del software diseña conjuntos de **condiciones de entrada** que ejerciten completamente todos los requisitos funcionales de un programa.
 - NO es una alternativa a las pruebas de caja blanca.
 - Complementan a las pruebas de caja blanca.
 - **Lo mejor es diseñar los casos de prueba usando los dos tipos de técnicas.**



➤ Pruebas de caja negra

- ✓ Intenta encontrar errores de las siguientes categorías:
 - Funciones incorrectas o ausentes.
 - Errores de interfaz.
 - Errores en estructuras de datos o en accesos a bases de datos externas.
 - Errores de rendimiento.
 - Errores de inicialización y de terminación.

➤ Pruebas de caja negra

✓ Tipos de pruebas:

- Prueba de los **valores límite**
 - Los errores suelen situarse en los límites.
 - Si la entrada se encuentra en el rango a..b entonces hay que probar con los valores a -1, a, a + 1, b - 1, b y b + 1
 - Si la entrada es un conjunto de valores entonces hay que probar con los valores max-1, max, max+1, min-1, min y min+1

Ejemplo de prueba de valores límites para esPrimo:

NUM	SalidaEsper	comentario
-1	no positivo	Valores límite: (num<=0) con num==-1
0	no positivo	Valores límite: (num<=0) con num==0
1	primo	Valores límite: (num<=0) con num==1
-1234123412341234	no positivo	Valores límite: NUM menor que el menor número entero
421342314321423142314	no primo	Valores límite: NUM mayor que el menor número entero

➤ Pruebas de caja negra

✓ Tipos de pruebas:

- Prueba de la **partición equivalente**.
 - Método de prueba de caja negra que divide el **dominio** de entrada de un programa en un conjunto de clases de datos de los que se pueden derivar casos de prueba.
 - Regularmente, una condición de entrada es un valor numérico específico, un rango de valores, un conjunto de valores relacionados o una condición lógica.
 - El diseño de estos casos de prueba para la partición equivalente se basa en la evaluación de las **clases de equivalencia**.
 - En otras palabras, consiste en analizar e identificar todos los conjuntos de clases de datos para ir tomando casos de pruebas que representen todas las posibilidades.

➤ Pruebas de caja negra

✓ Ejemplo:

- *Un sistema trabaja con contraseñas. Estas contraseñas son cadenas de hasta 6 caracteres que empiezan necesariamente por una letra y que puede contener letras, dígitos y el símbolo \$.*
- **¿Qué casos de prueba podríamos tomar para cubrir todas las posibilidades?**
 - “ “: Representa una entrada en blanco.
 - “4b2cd\$”: Representa una cadena que empieza por dígito y que sólo contiene letras, dígitos y el símbolo \$.
 - “4b;2c\$”: Representa una cadena que empieza por dígito y que contiene algún carácter que no es ni una letra ni un dígito ni el símbolo \$.
 - “\$ab4\$b”: Representa una cadena que no empieza por dígito y sólo contiene letras, dígitos y el símbolo \$.
 - “b\$\$;a5”: Representa una cadena que empieza por letra y contiene algún carácter que no es ni una letra ni un dígito ni el símbolo \$”.

➤ Pruebas de caja negra

- ✓ Prueba de la partición equivalente.
 - Las clases de equivalencia se pueden definir de acuerdo con las siguientes directrices:
 - Si un parámetro de entrada debe estar comprendido en un cierto **rango**, aparecen 3 clases de equivalencia: por debajo, en y por encima del rango.
 - Si una entrada requiere un **valor concreto**, aparecen 3 clases de equivalencia: por debajo, en y por encima del valor.
 - Si una entrada requiere un valor de entre los de un **conjunto**, aparecen 2 clases de equivalencia: en el conjunto o fuera de él.
 - Si una entrada es **booleana**, hay 2 clases: sí o no.
 - Si existe una razón para creer que el programa no trata de forma idéntica ciertos elementos pertenecientes a una clase, dividirla en clases de equivalencia menores.
 - Los mismos criterios se aplican a las **salidas** esperadas: hay que intentar generar resultados en todas y cada una de las clases.

➤ Pruebas de caja negra

✓ Prueba de la partición equivalente.

▪ Pasos:

- Una vez se tienen las clases válidas e inválidas definidas, se procede a definir los casos de pruebas.
- Para definir los casos de pruebas se tiene en cuenta lo siguiente:
 - Escribir un nuevo caso de cubra tantas clases de equivalencia **válidas** no cubiertas como sea posible hasta que todas las clases de equivalencia hayan sido cubiertas por casos de prueba.
 - Escribir un nuevo caso de prueba que cubra **una y sólo una clase de equivalencia inválida** hasta que todas las clases de equivalencias inválidas hayan sido cubiertas por casos de pruebas.

➤ **Proceso de depuración:**

- ✓ La depuración no es una prueba.
- ✓ Pero siempre ocurre como consecuencia de la prueba.
- ✓ Comienza con la ejecución de un caso de prueba.
- ✓ Se evalúan los resultados detectando el error.
- ✓ El proceso de depuración siempre tiene uno de los dos resultados siguientes:
 - Se encuentra la causa, se corrige y se elimina.
 - No se encuentra la causa.

Índice: Tema 6

6.1 Introducción

6.1.1 Conceptos básicos

6.1.2 Tipos de pruebas

6.2 Pruebas unitarias

6.2.1 Concepto

6.2.2 Framework JUnit

6.3 Sistemas de integración continua

6.3.1 Introducción

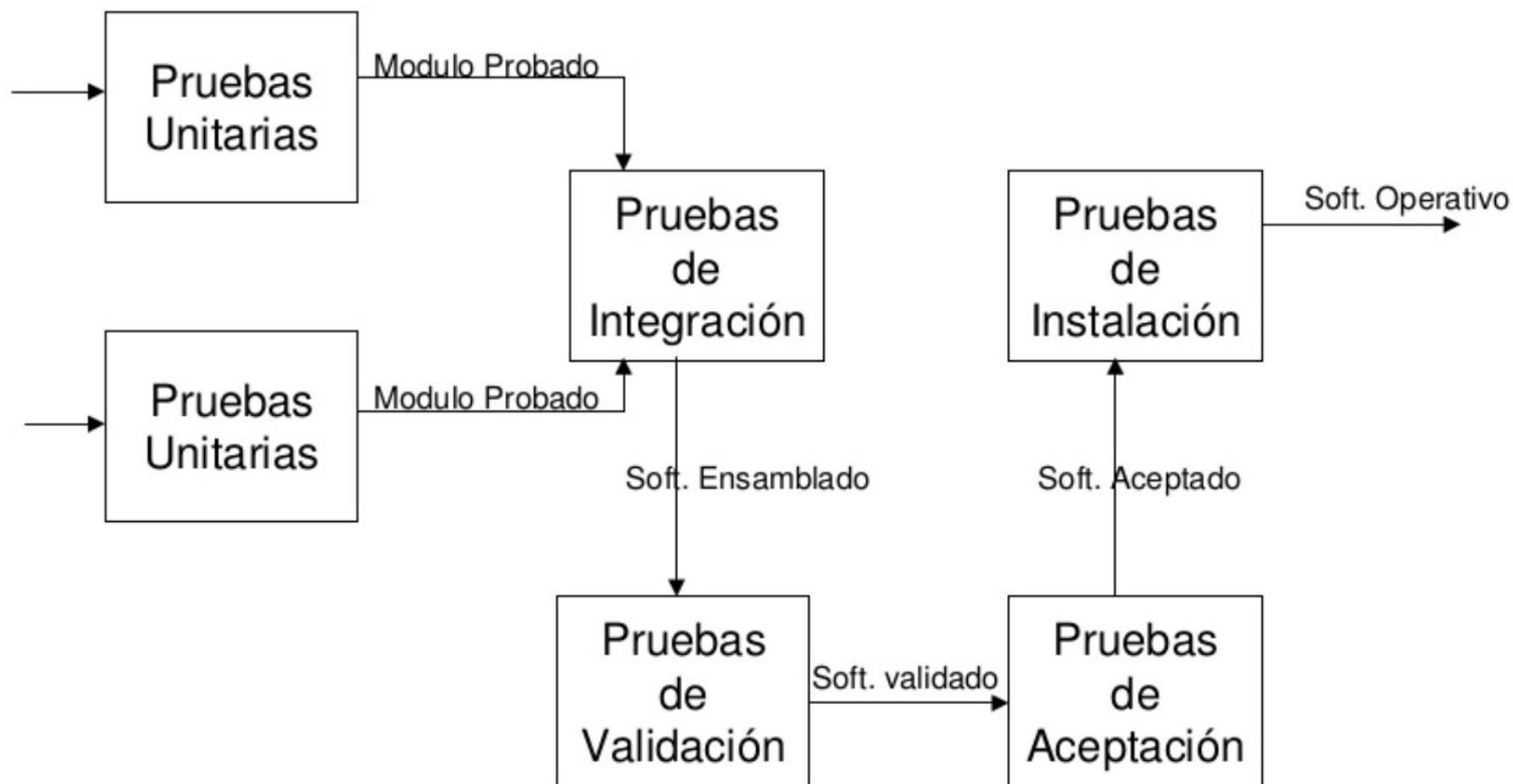
➤ Pruebas unitarias

- ✓ Es un procedimiento usado para validar que **un módulo o método de un objeto** fuente funciona apropiadamente y en forma independiente.
- ✓ A través de ellas se verifica que cierto módulo o método se ejecuta dentro de los **parámetros** y **especificaciones** concretadas en documentos tales como los casos de uso y el diseño detallado.
- ✓ Permiten detectar la inyección de **defectos** durante fases sucesivas de desarrollo o mantenimiento.
- ✓ Las pruebas unitarias típicamente son **automatizadas**, pero pueden llevarse a cabo de forma manual.
- ✓ Cuando son automatizadas es buena práctica que formen parte del **repositorio** que contiene al código probado.

➤ Pruebas unitarias

- ✓ Los datos de **entrada** son conocidos por el *Tester* o *Analista de Pruebas* y estos deben ser preparados con minuciosidad.
- ✓ Se debe conocer de antemano qué **resultados** debe devolver el componente según los datos de entrada utilizados en la prueba.
- ✓ Se deben **comparar** los datos obtenidos en la prueba con los datos esperados, si son idénticos podemos decir que el modulo superó la prueba y empezamos con la siguiente.
- ✓ Probar componentes implementados como unidades **individuales**
 - Prueba de especificación (de caja negra) que verifica el comportamiento observable **externamente**.
 - Prueba de estructura (de caja blanca) que verifica la implementación **interna**.

➤ PRUEBAS



➤ Frameworks

- ✓ Para llevar a cabo pruebas unitarias, existen un serie de *frameworks* que ofrecen un conjunto completo de utilidades, motores de ejecución y reportes.
- ✓ Entre los *frameworks* más empleados destacan:
 - XUnit: **JUnit**, NUnit, RUnit, PHPUnit...
 - TestNG
 - CPPUNIT
 - Visual Studio UnitTesting

Índice: Tema 6

6.1 Introducción

6.1.1 Conceptos básicos

6.1.2 Tipos de pruebas

6.2 Pruebas unitarias

6.2.1 Concepto

6.2.2 Framework JUnit

6.3 Sistemas de integración continua

6.3.1 Introducción

➤ Junit

- JUnit está diseñado alrededor de dos patrones de diseño principales: el patrón **Command** y el patrón **Composite**.
- Un **TestCase** es un objeto Command. Cualquier clase que contenga métodos de testeo debería extender la clase TestCase (JUnit 3). Un TestCase puede definir cualquier número de métodos públicos testXXX(). Cuando quieres comprobar el resultado esperado y el real, invocas una variante del método **assert()**.
- Las subclases de TestCase que contienen varios métodos testXXX() pueden utilizar los métodos **setUp()** y **tearDown()** para inicializar y liberar cualquier objeto común que se vaya a testear. Cada método de tests se ejecuta sobre su propia “instalación”, llamando a setUp() antes y a tearDown() después de cada método para asegurarse de que no hay efectos colaterales entre ejecuciones de tests.

➤ Clase de ejemplo

- ✓ Supóngase una clase con un método que tome un *array* de enteros como argumentos y devuelva el mayor valor encontrado en el *array*:

```
public class ClaseEjemplo {

    /**
     * Devuelve el elemento de mayor valor de una lista
     * @param list Un array de enteros
     * @return      El entero de mayor valor de la lista
     */

    public int funMaximo(int lista[]){
        ...
    }

}
```

➤ ¿Qué pruebas pueden hacerse?

- ✓ Caso normal: *array* con valores cualesquiera.
 - [3, 7, 9, 8] -> 9
- ✓ El mayor número se encuentra al principio o al final del *array*.
 - [9, 7, 8] -> 9
 - [8, 7, 9] -> 9
- ✓ El mayor número está duplicado en el *array*.
 - [9, 7, 9, 8] -> 9
- ✓ Sólo hay un elemento en el *array*.
 - [7] -> 7
- ✓ *Array* compuesto por números negativos.
 - [-4, -6, -7, -22] -> -4

➤ Procedimiento

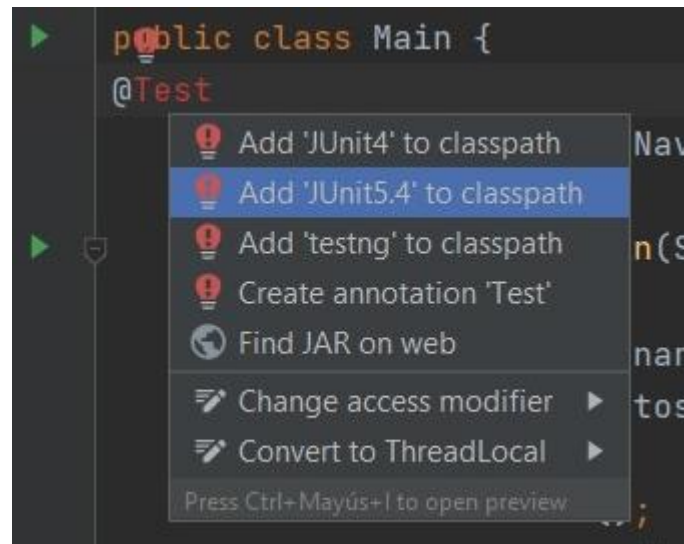
- ✓ Existen varias formas de ejecutar las pruebas, pero en general, pueden seguirse los pasos siguientes:
 - Importar las clases de JUNIT necesarias.
 - Definir la clase de pruebas.
 - Definir los métodos de prueba.
 - Serán ejecutados automáticamente por JUNIT.
- ✓ Otra opción es utilizar un IDE con JUNIT integrado.




➤ Procedimiento

✓ En IntelliJ:

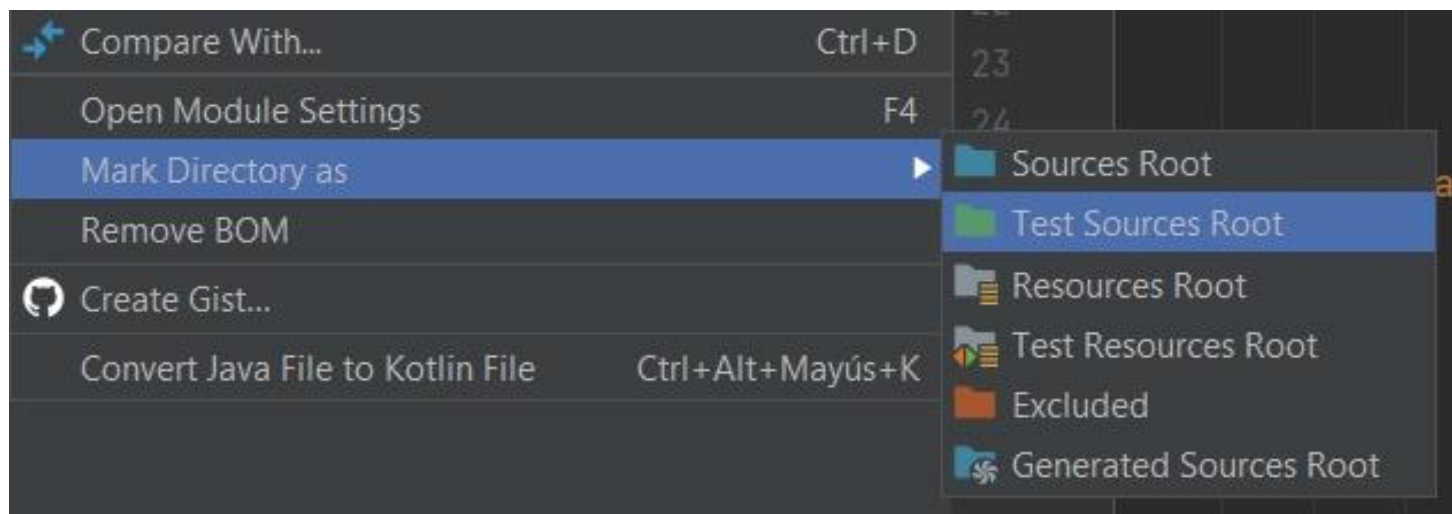
- ✓ Hay que añadir la librería de JUnit 5 al classpath. La manera más rápida:
 - ✓ Se escribe **@Test** en una parte del código y para resolver el problema IntelliJ ya nos da la opción de añadir JUnit.



➤ Procedimiento

✓ En IntelliJ:

- ✓ Tenemos que crear una carpeta con los Test. Se tiene que marcar el directorio como **Test Sources Root**.



➤ Procedimiento

✓ En IntelliJ:

- ✓ Creamos una clase Test para la clase que queramos probar. **Va a tener este aspecto:**

```
import org.junit.jupiter.api.*;

import static org.junit.jupiter.api.Assertions.*;

public class claseQueQuieroProbarTest {

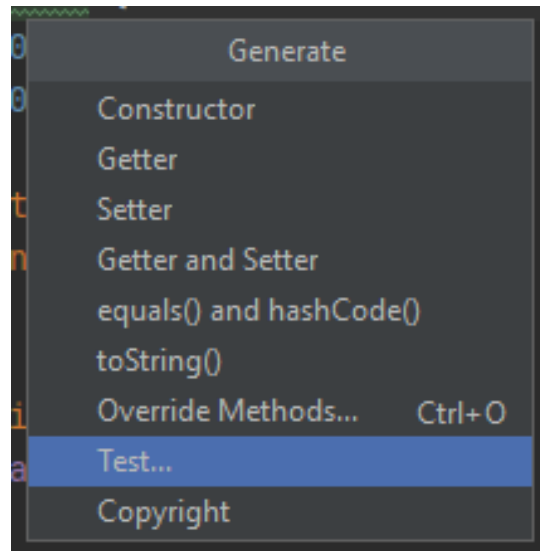
}
```

- ✓ Importante: Tiene que acabar en **Test**.

➤ Procedimiento

✓ En IntelliJ:

- ✓ Creamos una clase Test para la clase que queramos probar.
- ✓ La clase de test se puede generar de manera automática si pulsamos botón derecho sobre el nombre de la clase que queremos probar, seleccionamos “Generate” y luego “Test”.



➤ Procedimiento

✓ En IntelliJ:

- ✓ Creamos una clase Test para la clase que queramos probar.
- ✓ Podemos crear un TestSuite (una clase que prueba todas nuestras clases de prueba).

```
import org.junit.platform.runner.JUnitPlatform;
import org.junit.platform.suite.api.SelectClasses;
import org.junit.runner.RunWith;

@RunWith(JUnitPlatform.class)
@SelectClasses( { SumaTest.class } )

public class ProbandoTestSuite {
|
```

Aquí ponemos todas las clases de tipo TestCase que queremos probar, separadas por comas. En este caso sólo hay una case (SumaTest).

➤ Procedimiento

✓ En IntelliJ:

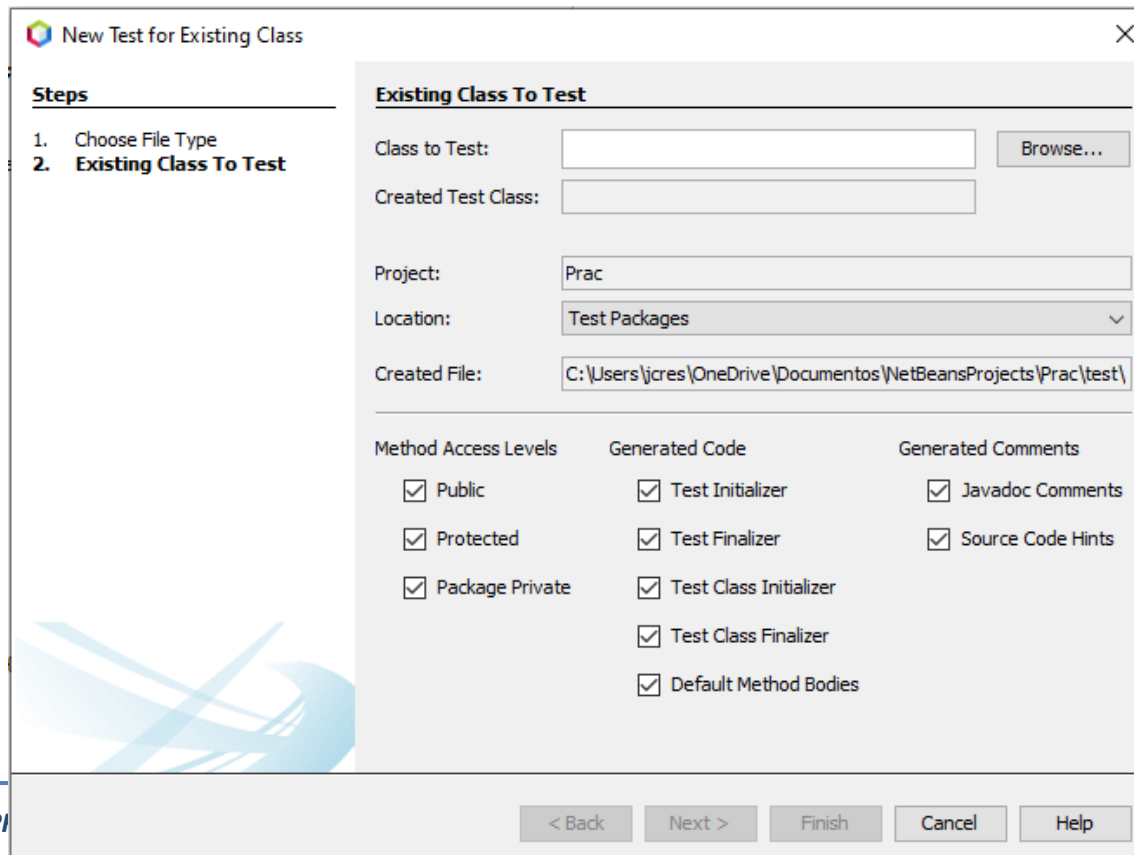
- ✓ Creamos una clase Test para la clase que queramos probar.
- ✓ Podemos crear un TestSuite (una clase que prueba todas nuestras clases de prueba).
- ✓ Para que funcione es posible que se necesite añadir algunos jar al proyecto que vienen aquí:

https://jar-download.com/maven-repository-class-search.php?search_box=org.junit.platform.runner.JUnitPlatform

➤ Procedimiento

✓ En NetBeans:

- ✓ Se hace click con el botón derecho del ratón sobre el nombre del proyecto - > new -> : Test for Existing Class...



New Test for Existing Class

Steps

1. Choose File Type
2. **Existing Class To Test**

Existing Class To Test

Class to Test: Browse...

Created Test Class:

Project:

Location:

Created File:

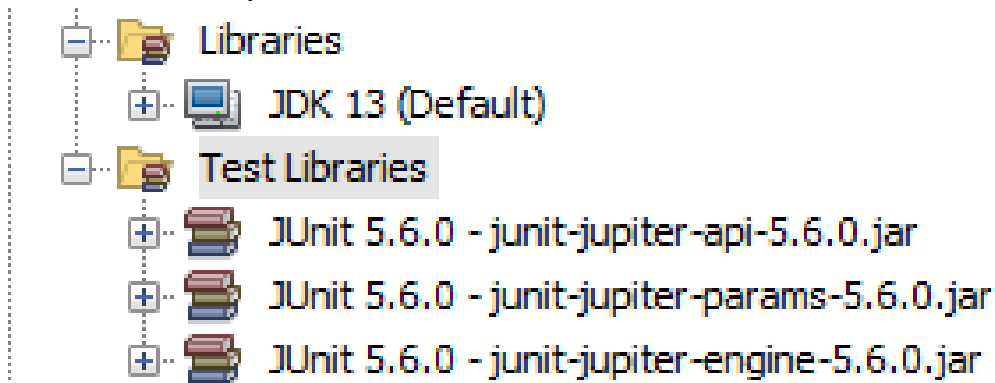
Method Access Levels	Generated Code	Generated Comments
☒ Public	☒ Test_INITIALIZER	☒ Javadoc Comments
☒ Protected	☒ Test_Finalizer	☒ Source Code Hints
☒ Package Private	☒ Test Class_INITIALIZER	
	☒ Test Class_Finalizer	
	☒ Default Method Bodies	

< Back Next > Finish Cancel Help

➤ Procedimiento

✓ En NetBeans:

- ✓ Automáticamente nos debería añadir las librerías de JUnit (si no, las añadimos nosotros).



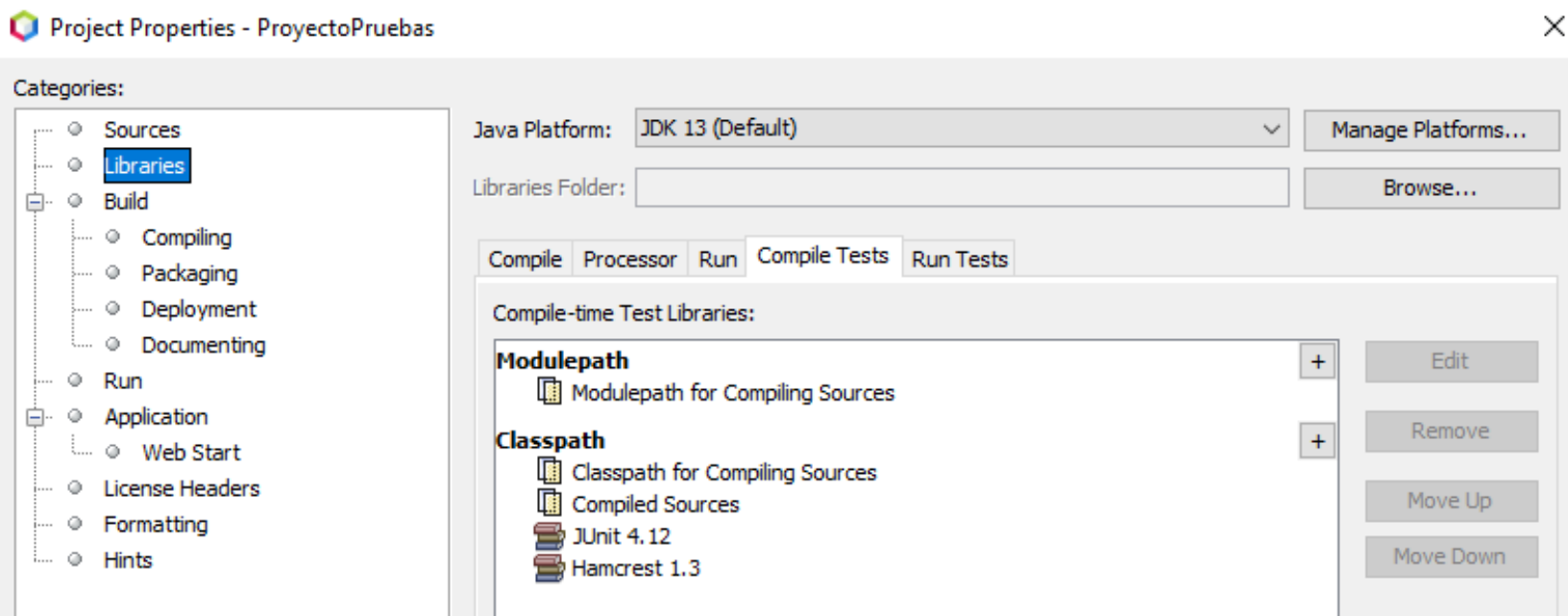
- ✓ Al haber creado el archivo Test, nos lo deja en una carpeta de Test. El nombre debe de ser “NombreDeLaClaseTest”. Si seleccionamos en “Browse...” la clase sobre la que queremos hacer el test, NetBeans coloca todo automáticamente con el nombre correcto (ver transparencia anterior).

➤ Procedimiento

- ✓ **En NetBeans:** Problema con algunas versiones de NetBeans y JUnit 5:

Vamos a usar JUnit 4.

Hay que modificar el classpath (botón derecho del ratón sobre el nombre del proyecto -> Properties -> Libraries). Hay que eliminar las referencias a JUnit 5 y añadir las de JUnit 4, sobre todo en “Compile Test”.



➤ Procedimiento

✓ En NetBeans:

- ✓ Creamos una clase Test para la clase que queramos probar. Se implementan todos los casos de prueba que se consideren necesarios.
- ✓ Se puede crear un **TestSuite** (una clase que prueba todas nuestras clases de prueba). Botón derecho del ratón sobre el nombre del proyecto -> New -> Test Suite... y se hace todo automáticamente.

Si se quiere añadir nuevas clases al Test Suite después de haberlo creado, se añaden en `@Suite.SuiteClasses` justo encima de la declaración de la clase. Por ejemplo, si tenemos dos clases en el proyecto que vamos a probar, la línea tendría este aspecto:

```
@Suite.SuiteClasses({nombrePaquete.NombreClase1.class, nombrePaquete.nombreClase2.class})
```

➤ **Anotaciones (1)**

✓ **@Before / @After**

- Significa que se ejecutará antes de cada método de test.

✓ **@BeforeClass / @AfterClass**

- Significa que se ejecutará antes/después de todos métodos de test.

✓ **@Ignore / @Disabled (JUnit 5)**

- Sirve para indicarle a *JUnit* que ignore el método de test.
- Esta anotación se utiliza generalmente cuando por alguna razón hemos modificado el código y estamos trabajando en el test.
- Puede ser que todavía nosotros no hayamos incluido código o que dependamos que algún desarrollador termine algo.

➤ **Anotaciones (2)**

✓ **@Test**

- Sirve para identificar que el método será un método a testear.

✓ **@Test(timeout)**

- Sirve para indicarle a *JUnit* que si el método tarda mas del tiempo esperado (en milisegundos) el test será marcado como fallido.
- Por tanto, es útil detectar procesos con mucha demora.

✓ **@Test(expected)**

- Sirve para indicar que el test debe lanzar algún tipo de excepción. En el caso que no se cumpla la condición el test será marcado como fallido.

➤ Comprobaciones (1)

✓ `assertEquals (valor_esperado, valor_real);`

- Los valores pueden ser de cualquier tipo.
- Si son *arrays*, no se comprueban elemento a elemento, sólo la referencia.

✓ `assertArrayEquals (valor_esperado[], valor_real[]);`

- Comprueba si los *arrays* son iguales (NO LA REFERENCIA).

✓ `assertTrue (condición_booleana)`

✓ `assertFalse (condición_booleana)`

- Comprueban si la condición booleana es cierta o falsa respectivamente.

➤ Comprobaciones (2)

- ✓ `assertSame` (Objeto esperado, Objeto real)
- ✓ `assertNotSame` (Objeto esperado, Objeto obtenido)
 - Comprueba si los objetos esperado y real son o no son la misma referencia.
- ✓ `assertNull` (Objeto)
- ✓ `assertNotNull` (Objeto objeto)
 - Comprueba si el objeto es o no es *Null*.
- ✓ `fail` (string Mensaje)
 - Imprime el mensaje y falla.
 - Útil para comprobar que se capturan excepciones.

➤ Clase de ejemplo

- ✓ Volvamos al ejemplo anterior:

```
public class ClaseEjemplo {

    public int funMaximo (int lista[]) {

        int indice, max = Integer.MAX_VALUE;
        for (indice = 0; indice < lista.length-1; indice++) {
            if (lista[indice] > max) {
                max = lista[indice];
            }
        }
        return max;
    }

}
```

➤ Código de Prueba

```
public class ClaseEjemploTest {

    /**
     * Test of fuMaximo method, of class ClaseEjemplo.
     */

    @Test
    public void testFunMaximo() {
        System.out.println("    * funMaximo");
        int[] lista = {3, 7, 8, 9};
        ClaseEjemplo instance = new ClaseEjemplo();
        int expResult = 9;
        int result = instance.funMaximo(lista);
        assertEquals(expResult, result);
    }

}
```

➤ Ejecutar Prueba

✓ ¿Qué está mal?

```
public static int funMaximo(int lista[]) {
    int indice, max = Integer.MAX_VALUE;
    for (indice = 0; indice < lista.length-1; indice++) {
        if (lista[indice] > max) {
            max = lista[indice];
        }
    }
    return max;
}
```

➤ Errores

✓ MIN_VALUE

✓ Analizar los extremos: Lista.Length

➤ Otras pruebas

```
public class ClaseEjemploTest extends TestCase {

    @Test
    public void testOrden() { ... }

    @Test
    public void testDuplicados() { ... }

    @Test
    public void testSoloUno() { ... }

    @Test
    public void testTodosNegativos() { ... }

}
```

➤ Otras anotaciones

```
public class ClaseEjemploTest extends TestCase {

    @BeforeClass
    public static void setUpClass() { ... }

    @AfterClass
    public static void tearDownClass() { ... }

    @Before
    public void setUp() { ... }

    @After
    public void tearDown() { ... }

}
```


Índice: Tema 6

6.1 Introducción

6.1.1 Conceptos básicos

6.1.2 Tipos de pruebas

6.2 Pruebas unitarias

6.2.1 Concepto

6.2.2 Framework JUnit

6.3 Sistemas de integración continua

6.3.1 Introducción

➤ Integración continua

- ✓ La integración continua es un modelo informático que consiste en hacer integraciones (compilación y ejecución de pruebas) automáticas de un proyecto lo más a menudo posible para así poder detectar fallos cuanto antes.
- ✓ El proceso consiste en programar una tarea que cada cierto tiempo:
 - Se descargue las fuentes desde el control de versiones.
 - Lo compile y ejecute pruebas.
 - Y genere informes.



Jenkins

sonarqube

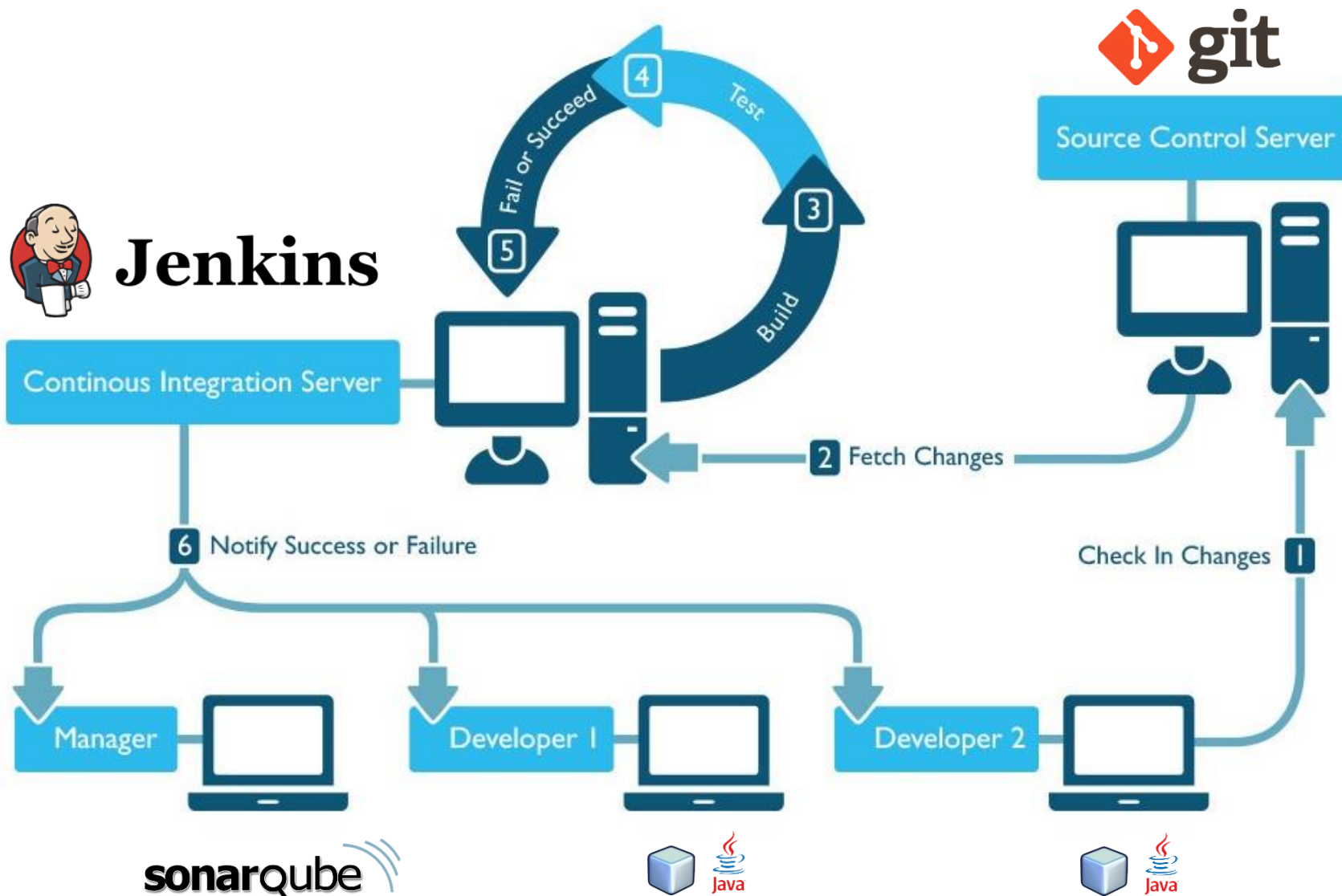
➤ Integración continua: Ventajas

- ✓ Los desarrolladores pueden detectar y solucionar problemas de **integración** de forma continua, evitando el caos de última hora cuando se acercan las fechas de entrega.
- ✓ Disponibilidad constante de una **versión** para pruebas, demos o lanzamientos anticipados.
- ✓ Ejecución inmediata de las pruebas unitarias.
- ✓ **Monitorización** continua de las métricas de calidad del proyecto.
- ✓ *A menudo la integración continua está asociada con las metodologías de **programación extrema** y **desarrollo ágil**.*



➤ Jenkins

- Es un servidor automatizado de integración continua gratuito y de código abierto.
- La base de Jenkins son las **tareas**, donde indicamos qué es lo que hay que hacer en un build.
- Es imprescindible tener un repositorio de **control de versiones**.
- Es capaz de **monitorizar** el control de versiones y actuar ante cualquier cambio.
- Los desarrolladores deben subir su trabajo **periódicamente** al repositorio. Además los proyectos tienen que tener un proceso de build automático



➤ Jenkins

Jenkins

Jenkins

- [New Job](#)
- [Manage Jenkins](#)
- [People](#)
- [Build History](#)
- [Redmine](#)
- [My Views](#)

Build Queue

No builds in the queue.

Build Executor Status

#	Status
1	Idle
2	Idle



[ENABLE AUTO REFRESH](#)

Continuous Integration Build Server.

[edit description](#)

All +						
S	W	Job ↓ ↓	Last Success	Last Failure	Last Duration	
		Project B	1 hr (#26)	2 hr (#24)	18 sec	
		Project A	1 mo 5 days (#31)	3 hr (#25)	4.2 sec	
		Project C	28 days (#32)	3 mo 0 days (#23)	3.5 sec	

Icon: [S](#) [M](#) [L](#)

Legend for all for failures for just latest builds

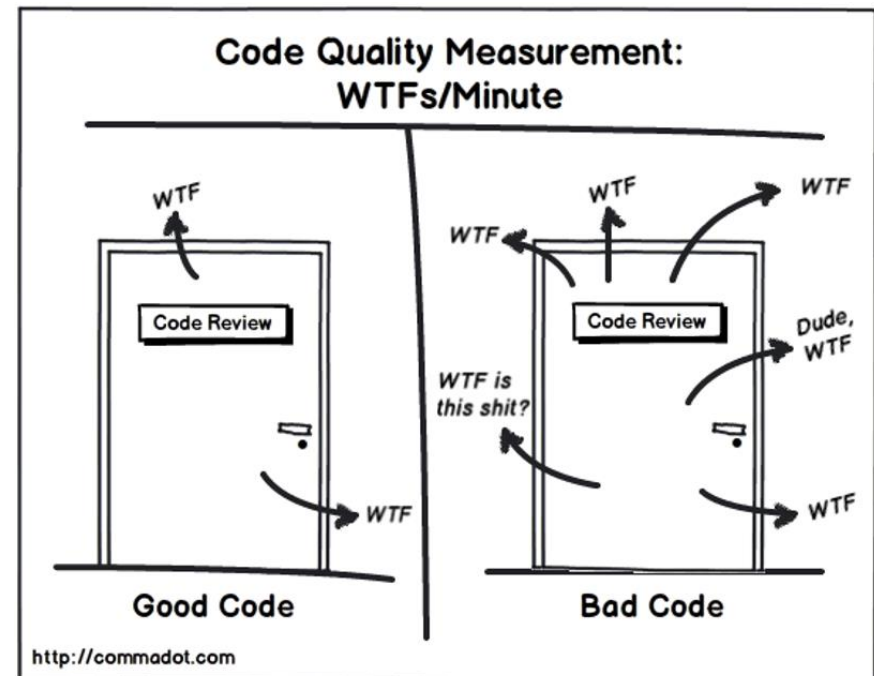
- **SonarQube**
- **SonarQube™** es un software open source que permite hacer **análisis estático** del código y obtener métricas.
- Emplea un algoritmo que dice si las expectativas de calidad que se fijaron durante el ciclo de vida del desarrollo se han cumplido y/o en cuanto tiempo se va a demorar en alcanzarlas.
- Permite conocer la **evolución**, analizando día a día lo que va sucediendo.

➤ SonarQube

- Entre las métricas emplean el código duplicado, code smells, bugs, vulnerabilidades, densidad de evidencias de seguridad, etc.

➤ “Code smells”:

- Método grande
- Clase grande (Objeto Dios)
- Demasiados parámetros
- Envidia de características
- Herencia rechazada
- Clase perezosa
- Complejidad artificial
- Identificadores excesivamente largos/cortos
- Supercallback
- Excesivo uso de literales



➤ SonarQube

